

OSSP SESSION WEEK-12

Task-1: Implement the Producer-Consumer problem using counting semaphores and POSIX threads.

Source code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 5

int buffer[SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 1; i <= 10; i++)
    {
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        printf("Produced: %d\n", i);
        in = (in + 1) % SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);
    }
    return NULL;
}

void *consumer(void *arg)
{
    for (int i = 1; i <= 10; i++)
    {
        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    }
    return NULL;
}

int main()
{
    pthread_t p, c;

    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

Output:

```
user@LAPX-0934-890:~$ nano producer_consumer.c
user@LAPX-0934-890:~$ gcc producer_consumer.c -o producer_consumer -pthread
user@LAPX-0934-890:~$ ./producer_consumer
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Produced: 5
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
Produced: 6
Produced: 7
Produced: 8
Produced: 9
Produced: 10
Consumed: 6
Consumed: 7
Consumed: 8
Consumed: 9
Consumed: 10
```

Observation: The Producer-Consumer problem was implemented using POSIX threads, counting semaphores, and mutex locks. The producer adds items to the buffer and the consumer removes them safely without accessing the buffer incorrectly.

Task-2: Design a deadlock scenario involving multiple threads competing for shared resources and prevent deadlock using resource ordering.

Soucre code:

```

GNU nano 8.7.1
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 1 locked Resource 1\n");

    sleep(1);

    pthread_mutex_lock(&resource2);
    printf("Thread 1 locked Resource 2\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 2 locked Resource 1\n");

    sleep(1);

    pthread_mutex_lock(&resource2);
    printf("Thread 2 locked Resource 2\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    return 0;
}

```

Output:

```

consumed. 10
user@LAPX-0934-890:~$ nano deadlock.c
user@LAPX-0934-890:~$ gcc deadlock.c -o deadlock -pthread
user@LAPX-0934-890:~$ ./deadlock
Thread 1 locked Resource 1
Thread 1 locked Resource 2
Thread 2 locked Resource 1
Thread 2 locked Resource 2

```

Observation: The program demonstrates how multiple threads compete for shared resources. By making both threads acquire Resource 1 first and Resource 2 second, resource ordering is maintained and circular waiting is avoided, preventing deadlock.